



Khawaja Collection

Premium Pakistani Fashion E-Commerce

Complete Build Specification & Claude Code Handoff

Prepared for: Claude Code (agentic build sessions)

Owner: Umair — Automa8

Version: 1.0

Date: 3 September 2026

Origin: Lovable prototype → Claude Code production build

Brand reference: facebook.com/KCofficial

How to use this document: Keep it in the repo root as `docs/BUILD-SPEC.pdf`. At the start of every Claude Code session, tell it: "Read `docs/BUILD-SPEC.pdf` and `CLAUDE.md` before doing anything." Then paste the phase prompt for the phase you are on (Section 17).

Table of Contents

1. Project Brief & Non-Negotiables — what we are building and what we must not do

2. Current State & Migration Path — Lovable → GitHub → Claude Code

3. Stack Decision — the one architectural call to make now

4. Repository Setup — exact commands

5. CLAUDE.md — paste-ready project memory file

6. Design System — tokens, type, spacing, motion

7. Routes & Information Architecture

8. Data Model & Database Schema

9. Folder Structure

10. Component Inventory & Contracts

11. Page Specifications — homepage, PLP, PDP, cart, checkout, search, account

12. State Management

13. SEO Implementation Plan

14. Performance Budget

15. Accessibility Requirements

16. Pakistan Commerce Rules — PKR, COD, sizing, shipping, provinces

17. Phase Plan & Paste-Ready Claude Code Prompts

18. Definition of Done — QA checklist per phase

19. Assets & Content Umair Must Supply

20. Guardrails — what Claude Code must never do in this repo

1. Project Brief & Non-Negotiables

1.1 What we are building

A production-quality, mobile-first e-commerce storefront for **Khawaja Collection (KC)**, a Pakistani fashion and clothing retailer. The site must support browsing, filtering, product discovery, wishlist, cart, and a checkout flow that is ready for Pakistani payment providers (Cash on Delivery first).

1.2 UX reference

LAAM (laam.com) is the reference for *experience patterns only*: category navigation depth, filter ergonomics, product-card density, the mega menu, and the general feel of a premium Pakistani fashion marketplace. It is a benchmark, not a template.

Hard rule — no copying. Do not reproduce LAAM's logo, brand name, copy, product photography, colour palette, class names, or DOM structure. Do not scrape their site. Do not fetch their assets at build or runtime. Every string, image, and colour in this project must be original to KC or a clearly-labelled placeholder.

1.3 Brand identity

Attribute	Value
Full name	Khawaja Collection
Short mark	KC
Category	Pakistani fashion — unstitched, ready-to-wear, bridal, menswear
Positioning	Premium but approachable. Editorial, not shouty.
Voice	Calm, confident, short sentences. No exclamation marks in UI copy.

Visual direction	Generous whitespace, large product imagery, restrained type, subtle gold accent
Facebook	facebook.com/KCoffical
Instagram / TikTok	PLACEHOLDER do not invent URLs — leave # until supplied

1.4 Success criteria

- Lighthouse mobile: Performance ≥ 90 , Accessibility ≥ 95 , SEO = 100.
- A product page is fully readable in "View Source" (server-rendered HTML, not an empty div).
- The mobile experience is a purpose-built layout, not a squeezed desktop grid.
- Zero hardcoded product data inside UI components — everything flows through the data layer.
- A non-technical person can add a product without touching React code.

2. Current State & Migration Path

2.1 Where the project stands

A first-pass UI exists as a Lovable project (preview URL: `id-preview--05847628-92dc-4c6a-a5dc-f928613a477d.lovable.app`). Lovable's default output is a **Vite + React + TypeScript + Tailwind + shadcn/ui** single-page application, usually with React Router and optionally a Supabase connection. Treat that project as a *visual reference and component donor*, not as the final architecture.

2.2 The migration in one line

Export Lovable → GitHub → clone locally → port the good components into a clean, SEO-capable app scaffolded and driven by Claude Code.

2.3 What to carry over from Lovable

Carry over	Leave behind
Tailwind theme values you liked (colours, radii, font choices)	Lovable-specific tooling, badges, and any <code>lovable-tagger</code> plugin
Presentational components: hero, product card, category card, footer	Page-level routing and data fetching — these get rebuilt
Any copy or headlines you already approved	Inline mock arrays scattered inside components
shadcn/ui primitives (button, dialog, sheet, drawer, select)	Anything that fetches from a client-only Supabase call on a page that needs SEO

3. Stack Decision

Recommendation: port to Next.js 15 (App Router) now, in Phase 0. Do it while the codebase is ten components, not sixty.

3.1 Why

This is a catalogue business. Its entire acquisition funnel is Google and Meta ads landing on category and product URLs. A Vite SPA ships an empty `<div id="root">` and renders everything client-side; product titles, prices, and structured data arrive after JavaScript executes. That is the single most expensive mistake to discover late — it is exactly the issue flagged in the Al-Madina Jewellers audit. Next.js gives server-rendered product pages, per-route metadata, ISR for price/stock freshness, and `next/image` responsive handling without extra plumbing.

The port is cheap right now because Tailwind classes, shadcn/ui components, and React presentational components move across almost unchanged. Only three things actually change: routing (file-based), image handling, and metadata.

3.2 Approved stack

Layer	Choice
-------	--------

Framework	Next.js 15, App Router, TypeScript strict mode
Styling	Tailwind CSS + CSS variables for design tokens
UI primitives	shadcn/ui (Radix under the hood) — accessible by default
Icons	lucide-react
Data (Phase 1–4)	Typed mock repository in <code>src/data/</code> behind a repository interface
Data (Phase 5+)	Supabase (Postgres + Storage + Auth) — same interface, swapped implementation
Client state	Zustand for cart + wishlist, persisted to localStorage
Forms	react-hook-form + zod
Toasts	sonner
Carousel	embla-carousel-react
Hosting	Vercel

3.3 If you decide to stay on Vite instead

Not recommended, but if the Lovable project is further along than expected: keep Vite, add `react-helmet-async` for per-route head tags, and add `vite-react-ssg` to pre-render every static route plus every product and category route at build time from the data layer. You must then rebuild and redeploy whenever the catalogue changes. Document this tradeoff in `CLAUDE.md` so it is a deliberate choice, not an accident.

4. Repository Setup

4.1 Get the Lovable code out

1. In Lovable: **GitHub → Connect to GitHub → Create Repository**. Name it `khawaja-collection`.
2. Clone it locally. Keep it somewhere **outside OneDrive** — OneDrive corrupts `.git` internals.
3. Create a branch `lovable-reference` and leave it untouched as a visual reference.

```
git clone https://github.com/<you>/khawaja-collection.git
cd khawaja-collection
git checkout -b lovable-reference
git push -u origin lovable-reference
git checkout main
```

4.2 Scaffold the production app

```
npx create-next-app@latest kc-store --typescript --tailwind --eslint --app --src-dir --import-alias "@/*"
cd kc-store
npx shadcn@latest init
npx shadcn@latest add button input sheet dialog drawer select accordion tabs \
  separator badge skeleton sonner label checkbox radio-group slider
npm i zustand embla-carousel-react react-hook-form zod @hookform/resolvers lucide-react
npm i -D @types/node
```

4.3 Environment variables

```
# .env.local — never commit this file
NEXT_PUBLIC_SITE_URL=https://khawajacollection.com
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=      # server-only, never prefixed NEXT_PUBLIC_
```

Commit a `.env.example` with empty values. Never commit `.env.local`. Never put a service-role key behind a `NEXT_PUBLIC_` prefix. No payment credentials anywhere in the repo at any point.

5. CLAUDE.md — Paste-Ready Project Memory

Create this file at the repo root before the first build session. Claude Code reads it automatically on every session.

```
# Khawaja Collection (KC) – Project Memory

## What this is
Premium Pakistani fashion e-commerce storefront. Mobile-first.
UX benchmark: LAAM. NEVER copy LAAM's assets, copy, colours, or markup.

## Stack
Next.js 15 App Router / TypeScript (strict) / Tailwind / shadcn-ui /
Zustand (cart + wishlist) / Supabase (Phase 5+) / Vercel.

## Commands
npm run dev          # local dev
npm run build        # must pass before any commit
npm run lint
npx tsc --noEmit     # must be clean

## Hard rules
1. No product data inside components. Everything via src/lib/repositories/*.
2. Every page exports `metadata` or `generateMetadata`. No exceptions.
3. Product and category pages are Server Components. Interactivity lives in
   small "use client" leaf components only.
4. Prices are integers in PKR. No floats. Format with formatPKR().
5. Use design tokens (CSS variables). No arbitrary hex values in JSX.
6. Every image uses next/image with width, height, sizes, and alt.
7. No new dependency without asking first.
8. Mobile layout is designed separately, not scaled down from desktop.
9. Do not invent social media URLs, phone numbers, addresses, or prices.
   Use PLACEHOLDER constants in src/config/site.ts.
10. Run `npm run build` before telling me a phase is done.

## Style
- Components: PascalCase files in src/components/<domain>/
- Hooks: use*.ts in src/hooks/
- Types: src/types/index.ts – single source of truth
- Prefer composition over props explosion. Max ~7 props per component.

## Definition of done for any task
TypeScript clean, build passes, responsive at 360 / 768 / 1024 / 1440,
keyboard navigable, no console errors, no layout shift on image load.
```

6. Design System

6.1 Colour tokens

Define once in `src/app/globals.css` as CSS variables, expose through `tailwind.config.ts`. Never write a raw hex in a component.

```
:root {
  --kc-white:      #FFFFFF;
  --kc-paper:      #FAF9F7; /* off-white page background */
  --kc-sand:       #F1ECE4; /* soft beige section blocks */
  --kc-line:       #E4E0D9; /* hairline borders */
  --kc-ink:        #14110F; /* near-black, primary text */
  --kc-charcoal:   #3A3633; /* secondary text */
  --kc-muted:      #8A837C; /* tertiary text, meta */
  --kc-gold:       #B08D3F; /* accent – used sparingly */
  --kc-gold-soft:  #E8DCC2;
  --kc-sale:       #A32E2E; /* sale price + discount badge only */
  --kc-success:    #2C7A4B;
}
```

Token	Allowed use
--kc-gold	Micro-accents only: active nav underline, badge borders, focus ring, small rule marks. Never a full button fill, never large areas.
--kc-sale	Sale price text and discount percentage badge. Nothing else.
--kc-ink	Primary buttons, headings, body text.
--kc-sand	Alternating section backgrounds to break up long white pages.

6.2 Typography

Two families, loaded via `next/font` with `display: swap`.

- **Display / headings:** a high-contrast serif — Cormorant Garamond or Playfair Display. Used for hero headlines, section titles, editorial banners.
- **UI / body:** a neutral grotesque — Inter or Geist. Used for everything else: nav, buttons, prices, product names, forms.

Role	Mobile	Desktop	Weight / tracking
Hero headline	34px	64px	Serif 400, tracking -1%
Section title	22px	32px	Serif 400
Editorial eyebrow	11px	12px	Sans 500, uppercase, tracking +14%
Product name	13px	14px	Sans 400
Price	13px	14px	Sans 600, tabular-nums
Body	14px	15px	Sans 400, line-height 1.6
Button label	12px	12px	Sans 500, uppercase, tracking +8%

6.3 Spacing, radius, elevation

- Spacing scale: 4 / 8 / 12 / 16 / 24 / 32 / 48 / 64 / 96 px. Nothing off-scale.
- Section vertical rhythm: 48px mobile, 96px desktop.
- Page gutter: 16px mobile, 24px tablet, 40px desktop, max content width 1440px.
- Radius: 0px on product imagery and cards (editorial, square), 4px on buttons and inputs, 8px on drawers and modals. Resist rounding everything — it cheapens the look.
- Shadows: almost none. One soft shadow token for drawers and sticky bars only. Depth comes from hairline borders and whitespace.

6.4 Imagery rules

- Product card image ratio: **3:4** — enforced everywhere, no exceptions.
- PDP gallery ratio: **3:4**, editorial banners **16:9** desktop / **4:5** mobile.

- Category card: **4:5**.
- Every image: explicit width/height, `sizes` attribute, `loading="lazy"` except the LCP hero which uses `priority`.
- Placeholder strategy: neutral beige blocks with the KC monogram, generated locally. Never hotlink stock or competitor images.

6.5 Motion

Interaction	Behaviour
Product card hover (desktop)	Cross-fade to second image, 300ms ease-out. No zoom, no lift.
Button hover	Background shifts to charcoal, 150ms.
Drawer / sheet	Slide 250ms cubic-bezier(0.32, 0.72, 0, 1).
Add to cart	Cart icon count pops once (scale 1 → 1.15 → 1, 200ms) + toast.
Wishlist	Heart fills, single 180ms scale pulse.
Page / section entry	Nothing. No scroll-triggered fade-ins. They hurt LCP and feel dated.
Reduced motion	Respect <code>prefers-reduced-motion</code> : disable all transitions.

7. Routes & Information Architecture

Route	Rendering	Notes
<code>/</code>	Static + ISR 1h	Homepage
<code>/new-arrivals</code>	ISR	PLP preset filter
<code>/women</code> , <code>/men</code>	ISR	Top-level category PLPs
<code>/women/[subcategory]</code>	ISR	e.g. <code>/women/unstitched</code>
<code>/unstitched</code> , <code>/ready-to-wear</code> , <code>/bridal</code>	ISR	Cross-cutting category PLPs
<code>/sale</code>	ISR 15m	Distinct sale styling
<code>/collections/[slug]</code>	ISR	e.g. <code>/collections/summer-collection</code>
<code>/products/[slug]</code>	ISR 15m + <code>generateStaticParams</code>	PDP. SEO-critical.
<code>/search</code>	Client	Query param <code>?q=</code>
<code>/cart</code>	Client	Full cart page (drawer is the primary UI)
<code>/checkout</code>	Client	<code>noindex</code>
<code>/wishlist</code>	Client	<code>noindex</code>
<code>/track-order</code>	Client	
<code>/account</code> + <code>/login</code> <code>/register</code> <code>/forgot-password</code>	Client	<code>noindex</code> . Sub-routes: profile, orders, orders/[id], addresses
<code>/about</code> , <code>/contact</code> , <code>/faqs</code> , <code>/shipping</code> , <code>/returns</code> , <code>/privacy</code> , <code>/terms</code> , <code>/refund-policy</code>	Static	Content pages from MDX or a simple content map

Slug rules: lowercase, hyphenated, no stop words dropped, never include the ID. Example: `/products/embroidered-lawn-3-piece-suit` . Slugs are immutable once published — if a product is renamed, keep the old slug and add a 301 in `next.config.ts` .

8. Data Model & Database Schema

8.1 TypeScript types — `src/types/index.ts`

```

export type Currency = "PKR";

export interface ProductImage {
  url: string;
  alt: string;
  width: number;
  height: number;
  isPrimary?: boolean;
}

export interface ProductVariant {
  id: string;
  sku: string;
  size: string;          // "XS" | "S" ... or "Unstitched"
  colorName: string;     // "Ivory"
  colorHex: string;      // "#F3EFE7"
  stock: number;         // 0 = out of stock
  priceOverride?: number;
}

export interface Product {
  id: string;
  slug: string;
  name: string;
  description: string;    // rich text / markdown
  shortDescription: string; // max 160 chars, also used for meta
  price: number;          // integer PKR
  salePrice?: number;     // integer PKR, must be < price
  categorySlug: string;   // "women"
  subcategorySlug?: string; // "unstitched"
  collectionSlugs: string[];
  images: ProductImage[];
  variants: ProductVariant[];
  fabric?: string;
  pieces?: number;        // 1, 2, 3 piece
  care?: string;
  tags: string[];
  rating: number;         // 0-5
  reviewCount: number;
  isFeatured: boolean;
  isNewArrival: boolean;
  isBestSeller: boolean;
  isOnSale: boolean;
  createdAt: string;      // ISO
}

export interface Category {
  slug: string; name: string; description?: string;
  image?: ProductImage; parentSlug?: string; sortOrder: number;
}

export interface Collection {
  slug: string; name: string; tagline?: string;
  heroImage?: ProductImage; isActive: boolean;
}

export interface CartLine {
  productId: string; variantId: string; slug: string;
  name: string; image: string; size: string; colorName: string;
  unitPrice: number; // resolved: salePrice ?? price
  quantity: number;
}

```

8.2 Repository interface — the seam that makes swapping trivial

```

// src/lib/repositories/product-repository.ts
export interface ProductQuery {
  category?: string; subcategory?: string; collection?: string;
  sizes?: string[]; colors?: string[]; fabrics?: string[];
  minPrice?: number; maxPrice?: number;
  onSale?: boolean; inStockOnly?: boolean; minDiscount?: number;
  sort?: "featured" | "newest" | "price-asc" | "price-desc" | "best-selling";
  page?: number; perPage?: number; q?: string;
}

export interface ProductRepository {
  list(query: ProductQuery): Promise<{ items: Product[]; total: number; facets: Facets }>;
  getBySlug(slug: string): Promise<Product | null>;
  getAllSlugs(): Promise<string[]>;
  related(product: Product, limit: number): Promise<Product[]>;
}

```

Phase 1-4: `MockProductRepository` reads from `src/data/products.ts`. **Phase 5:** `SupabaseProductRepository` implements the same interface. Components never import either directly — they import from `src/lib/repositories/index.ts`, which picks one based on an env flag.

8.3 Supabase schema (Phase 5)

```
create table categories (
  slug text primary key,
  name text not null,
  parent_slug text references categories(slug),
  description text,
  image_url text,
  sort_order int default 0
);

create table collections (
  slug text primary key,
  name text not null,
  tagline text,
  hero_image_url text,
  is_active boolean default true
);

create table products (
  id uuid primary key default gen_random_uuid(),
  slug text unique not null,
  name text not null,
  description text,
  short_description text,
  price integer not null check (price > 0),
  sale_price integer check (sale_price is null or sale_price < price),
  category_slug text references categories(slug),
  subcategory_slug text references categories(slug),
  fabric text, pieces int, care text,
  tags text[] default '{}',
  rating numeric(2,1) default 0,
  review_count int default 0,
  is_featured boolean default false,
  is_new_arrival boolean default false,
  is_best_seller boolean default false,
  is_active boolean default true,
  created_at timestamptz default now()
);

create table product_images (
  id uuid primary key default gen_random_uuid(),
  product_id uuid references products(id) on delete cascade,
  url text not null, alt text, sort_order int default 0,
  is_primary boolean default false
);

create table product_variants (
  id uuid primary key default gen_random_uuid(),
  product_id uuid references products(id) on delete cascade,
  sku text unique not null,
  size text not null, color_name text, color_hex text,
  stock int default 0 check (stock >= 0),
  price_override integer
);

create table product_collections (
  product_id uuid references products(id) on delete cascade,
  collection_slug text references collections(slug) on delete cascade,
  primary key (product_id, collection_slug)
);

create table orders (
  id uuid primary key default gen_random_uuid(),
  order_number text unique not null, -- KC-2026-00042
  user_id uuid references auth.users(id),
  email text, phone text not null,
  ship_name text, ship_line1 text, ship_line2 text,
  ship_city text, ship_province text, ship_postal text,
  subtotal integer, shipping integer, discount integer, total integer,
  payment_method text check (payment_method in ('cod','card','bank_transfer')),
  payment_status text default 'pending',
  status text default 'placed'
  check (status in ('placed','confirmed','processing','shipped','out_for_delivery','delivered','cancelled')),
  created_at timestamptz default now()
);

create table order_items (
```

```

id uuid primary key default gen_random_uuid(),
order_id uuid references orders(id) on delete cascade,
product_id uuid, variant_id uuid,
name_snapshot text, size text, color_name text,
unit_price integer, quantity int
);

create table wishlists (
  user_id uuid references auth.users(id) on delete cascade,
  product_id uuid references products(id) on delete cascade,
  created_at timestamptz default now(),
  primary key (user_id, product_id)
);

-- Indexes
create index on products (category_slug, is_active);
create index on products (created_at desc);
create index on product_variants (product_id);
create index on orders (order_number);

```

RLS: enable Row Level Security on every table. Public `select` on products, images, variants, categories, collections where `is_active`. Orders: readable only by owning `user_id`, or via a server-side lookup by order number + phone for guest tracking. Never expose a table that lets an anonymous user read all orders.

9. Folder Structure

```

src/
  app/
    (shop)/
      page.tsx                # homepage
      women/page.tsx
      women/[subcategory]/page.tsx
      men/page.tsx
      unstitched/page.tsx
      bridal/page.tsx
      ready-to-wear/page.tsx
      new-arrivals/page.tsx
      sale/page.tsx
      collections/[slug]/page.tsx
      products/[slug]/page.tsx
      search/page.tsx
    (account)/
      login/ register/ forgot-password/ account/...
    cart/page.tsx
    checkout/page.tsx
    track-order/page.tsx
    (content)/about|contact|faqs|shipping|returns|privacy|terms/page.tsx
    api/
      newsletter/route.ts
      orders/route.ts
    sitemap.ts
    robots.ts
    layout.tsx
    globals.css
  components/
    layout/      Header AnnouncementBar MegaMenu MobileNav Footer
    product/     ProductCard ProductGrid ProductCarousel ProductGallery
                  ProductInfo SizeSelector ColorSelector QuickViewDialog
                  QuickAddButton WishlistButton PriceBlock ProductBadge
    catalog/     FilterSidebar FilterDrawer ActiveFilters SortDropdown
                  Pagination Breadcrumbs CategoryCard
    cart/        CartDrawer CartItem CartSummary StickyAddToCart
    search/      SearchModal SearchSuggestions SearchResults
    home/        HeroSection ShopByCategory EditorialBanner
                  SaleSection SocialGrid Newsletter
    ui/          shadcn primitives
  lib/
    repositories/ product-repository.ts mock/ supabase/ index.ts
    supabase/     client.ts server.ts
    format.ts     formatPKR discountPercent
    seo.ts        buildMetadata jsonLd helpers
    utils.ts
  hooks/         useCart useWishlist useRecentlyViewed useMediaQuery
  store/         cart-store.ts wishlist-store.ts ui-store.ts
  data/          products.ts categories.ts collections.ts (mock, Phase 1-4)
  config/        site.ts nav.ts filters.ts
  types/         index.ts

```

10. Component Inventory & Contracts

10.1 ProductCard — the most reused component in the app

```
interface ProductCardProps {
  product: Product;
  priority?: boolean;           // true only for first row above the fold
  showQuickAdd?: boolean;      // default true on desktop, false on mobile
  className?: string;
}
```

Renders: primary image in a fixed 3:4 frame; second image cross-fades on desktop hover; wishlist heart top-right (36px tap target, stops event propagation); badge stack top-left (**NEW** **-30%**); below the image — category eyebrow, product name clamped to two lines, then **PriceBlock**. Quick View appears on hover as a centred overlay bar on desktop only. Quick Add opens a compact size picker if the product has multiple sizes; adds directly if there is only one.

Rules: the whole card is a single `<Link>`; heart and quick-add are buttons layered above it with `preventDefault`. Card must never shift layout when the image loads. Out-of-stock cards show a muted overlay and disable Quick Add.

10.2 Other contracts (abbreviated)

Component	Key props / behaviour
Header	Sticky, transparent over hero on homepage only, solid on scroll past 80px. Hides on scroll-down / reveals on scroll-up on mobile.
MegaMenu	<code>{ section: NavSection }</code> . Opens on hover intent (120ms delay) and on focus. Three columns of links plus one promotional image tile. Escape closes; focus trapped while open.
MobileNav	Full-height sheet. Accordion category tree, max two levels deep. Account and wishlist links pinned to the bottom.
ProductGrid	<code>{ products, columns?: {mobile:2, tablet:3, desktop:4} }</code> . Renders <code>ProductCardSkeleton</code> while loading.
FilterSidebar / FilterDrawer	Same state, two presentations. Filters write to URL search params — filtered views must be shareable and back-button safe. Drawer shows a live "Show 42 results" button.
ProductGallery	Desktop: vertical thumbnail rail + main image with click-to-zoom. Mobile: swipeable full-bleed carousel with dot indicators. Never a lightbox library.
SizeSelector	Pills, not a dropdown. Unavailable sizes are struck through and disabled, not hidden.
CartDrawer	Right sheet, 420px desktop / full-width mobile. Free-delivery progress bar at top. Empty state with a category CTA.
StickyAddToCart	Mobile PDP only. Appears once the main Add to Cart scrolls out of view. Shows price + size chip + button.
SearchModal	Command-palette style. Opens on click or <code>/</code> key. Debounced 200ms. Sections: recent, popular, categories, products (max 6 with thumbnails).

11. Page Specifications

11.1 Homepage — section order and intent

1. **Announcement bar** — rotating messages, 3 max, from `src/config/site.ts`: free delivery above PKR 5,000; cash on delivery available; easy exchange. Dismissible, remembered in localStorage for the session.
2. **Hero** — full-bleed editorial image, 88vh mobile / 92vh desktop. Eyebrow, serif headline, one supporting line, two CTAs: *Shop Women* (filled) and *Shop Men* (outline). This image is the LCP element: `priority`, preloaded, correctly sized. Text sits on a subtle bottom gradient, never on a coloured box.
3. **Shop by Category** — six 4:5 cards: Women, Men, Unstitched, Ready to Wear, Bridal, Accessories. Horizontal scroll on mobile, 3x2 grid on desktop.
4. **New Arrivals** — carousel, 8 products, "View All" link to `/new-arrivals`.
5. **Featured collection** — full-width editorial split. "The New Season" + CTA.
6. **Trending Now** — 8-product grid, driven by `isBestSeller`.
7. **Women's edit** — banner + 4 products.
8. **Men's edit** — banner + 4 products.
9. **Sale** — visually distinct: `--kc-ink` background, white type, discount badges prominent. 6 products + "Shop All Sale".
0. **Follow Khawaja Collection** — 6 square social tiles, Facebook + Instagram links. Instagram URL stays a placeholder until supplied.
1. **Newsletter** — "Join the KC Family". Email input + Subscribe. One line of privacy-friendly copy. Posts to `/api/newsletter`.
2. **Footer** — four link columns (Customer Care, About KC, Shop, Legal), social row, payment/delivery icons, copyright.

11.2 Product Listing Page

Breadcrumbs → H1 category title → one-line category description → result count and sort row. Desktop: 280px sticky filter sidebar + 4-column grid. Tablet: 3 columns, filters in a drawer. Mobile: 2 columns, a sticky bottom bar with *Filter* and *Sort* buttons.

- Filters: category, subcategory, size, colour (swatches), price range, collection, availability, discount band, fabric.
- All filter state lives in the URL: `/women?size=M,L&color=ivory&sort=price-asc&page=2`.
- Applied filters render as removable chips above the grid, plus "Clear all".
- Pagination: numbered pages, not infinite scroll — better for SEO and for users on unreliable mobile data. Include `rel="next"/"prev"`.
- Empty state: a clear message, the active filters listed, and a one-tap clear.

11.3 Product Detail Page

Two-column on desktop (gallery 58% / info 42%, info column sticky). Single column on mobile with a full-bleed gallery.

Info column order: category eyebrow → H1 name → rating + review count → PriceBlock (sale price, struck original, discount badge) → short description → colour swatches → size pills + "Size Guide" link (opens a dialog) → quantity stepper → *Add to Cart* (primary) and *Buy Now* (secondary) → wishlist link → trust row (COD available, easy exchange, delivery estimate).

Below the fold: accordion with Description, Details & Fabric, Size Guide, Shipping Information, Exchange & Return. Then *You May Also Like* (8 items) and *Recently Viewed* (localStorage, max 12, excludes the current product).

Mobile adds `StickyAddToCart`. Selecting an unavailable size disables the button with inline text rather than a toast.

11.4 Cart

Drawer is the primary surface; `/cart` is the same data in a full-page layout. Each line: 3:4 thumbnail, name (links to PDP), size, colour, quantity stepper, line total, remove, save-for-later. Summary: subtotal, shipping (free above PKR 5,000, otherwise flat rate from config), discount, total. Buttons: *View Cart* and *Checkout*. Removing a line offers an undo toast for 5 seconds.

11.5 Checkout

Single page, three stacked sections, with an order summary sticky on the right (desktop) or collapsed at the top (mobile).

1. **Contact** — email (optional), phone (required, validated as a Pakistani mobile).
2. **Shipping** — full name, address line 1, line 2, city, province (select), postal code (optional in PK), delivery notes.
3. **Payment** — Cash on Delivery (default, enabled), Card and Bank Transfer rendered but marked *Coming soon* and disabled.

```
// src/lib/payments/types.ts – the seam for later integration
export interface PaymentProvider {
  id: "cod" | "card" | "bank_transfer";
  label: string;
  isEnabled: boolean;
  initiate(order: OrderDraft): Promise<PaymentInitResult>;
  verify(reference: string): Promise<PaymentStatus>;
}
```

Implement `CodProvider` fully and `MockCardProvider` as a stub that throws "not configured". Real providers (JazzCash, Easypaisa, PayFast, Safepay) plug in later without touching the UI. **No credentials in the repo, ever.** Order creation happens in a server route, never from the client.

11.6 Search, Wishlist, Account, Track Order

- **Search:** modal for quick lookup, `/search?q=` for a full PLP-style result page with the same filters. Match on name, category, collection, tags, fabric. Recent searches in `localStorage` (max 6); popular searches from config.
- **Wishlist:** guests get a `localStorage` list; on login, merge the local list into the account list rather than overwriting. "Move to cart" needs a size — open the size picker.
- **Account:** Supabase Auth, email + password to start. Pages: profile, orders, order detail, addresses, wishlist. All `noindex`.
- **Track Order:** order number + phone or email. Server-side lookup only. Six-step visual timeline: Placed → Confirmed → Processing → Shipped → Out for Delivery → Delivered. Never expose order data from a client query.

12. State Management

Concern	Where it lives
Cart	Zustand store, persisted to <code>localStorage</code> under <code>kc-cart-v1</code> . Line key = <code>productId:variantId</code> . Re-validate prices and stock server-side at checkout — never trust the stored price.
Wishlist	Zustand + <code>localStorage</code> <code>kc-wishlist-v1</code> ; synced to Supabase when authenticated.
Recently viewed	<code>localStorage</code> <code>kc-recent-v1</code> , capped at 12 slugs.
UI (drawers, modals)	Small <code>ui-store</code> — cart drawer open, search modal open, mobile nav open. Only one may be open at a time.
Filters & sort	URL search params. Never component state.
Server data	Server Components + <code>fetch</code> caching. No client data-fetching library needed in Phase 1–5.

Hydration: persisted Zustand stores must not render different markup on server and client. Gate cart-count and wishlist-heart rendering behind a mounted check, or use `skipHydration` and `rehydrate` in an effect. Getting this wrong produces intermittent hydration errors that are painful to trace later.

13. SEO Implementation Plan

- Every route exports `metadata` or `generateMetadata`: title, description, canonical, Open Graph (with a 1200×630 image), Twitter card.
- Title pattern: `{Product Name} | Khawaja Collection` — category: `{Category} Collection | Khawaja Collection`. Under 60 characters.
- Meta description from `shortDescription`, 150–160 characters, never duplicated.
- JSON-LD on every relevant page: `Organization` + `WebSite` (with `SearchAction`) in the root layout; `Product` with `Offer` (price, PKR, availability) and `AggregateRating` on PDPs; `BreadcrumbList` on PDP and PLP; `ItemList` on category pages.
- `app/sitemap.ts` generated from the repository: all static routes + every product, category, and collection slug, with `lastModified`.
- `app/robots.ts`: allow all, disallow `/checkout`, `/account`, `/cart`, `/wishlist`, `/api`. Point to the sitemap.
- Filtered PLP URLs: canonical points to the unfiltered category URL to avoid duplicate content. Paginated pages are self-canonical.
- One `<h1>` per page. Descriptive alt text on every product image (`"{product name} – {colour} – front view"`), never "product image".

- Add Google Search Console + a sitemap submission to the launch checklist.

14. Performance Budget

Metric (mobile, 4G throttled)	Budget
Largest Contentful Paint	< 2.5s
Cumulative Layout Shift	< 0.05
Interaction to Next Paint	< 200ms
Initial JS transferred (homepage)	< 180 KB gzipped
Hero image transferred	< 180 KB (AVIF/WebP, responsive srcset)
Fonts	2 families, subset to latin, <code>display: swap</code> , self-hosted via next/font

- Server Components by default. `"use client"` only on leaf components that need state or event handlers.
- Dynamic-import heavy client pieces: CartDrawer, SearchModal, FilterDrawer, QuickViewDialog, the size-guide dialog.
- Every image reserves its space via aspect-ratio — CLS from product grids is the most common failure in stores like this.
- No carousel library on the homepage above the fold; the hero is a static image.
- Run `npx @next/bundle-analyzer` at the end of each phase and record the numbers.

15. Accessibility Requirements

- Keyboard: full traversal of nav, mega menu, filters, gallery, and checkout. Visible focus ring using `--kc-gold` at 2px offset.
- Focus trapping and restore for every drawer, sheet, and dialog. Escape always closes.
- Colour contrast: body text $\geq 4.5:1$. `--kc-muted` on white is borderline — never use it for anything smaller than 13px or for essential information.
- Tap targets $\geq 44 \times 44$ px on mobile. The wishlist heart is the usual offender.
- Colour swatches carry a visible or accessible colour name — colour alone must never be the only way to distinguish a variant.
- Cart and wishlist changes announce via an `aria-live="polite"` region, not only a toast.
- Form errors are tied to inputs with `aria-describedby`, and are text, not just red borders.
- All images have meaningful alt text; decorative editorial images use `alt=""`.

16. Pakistan Commerce Rules

Concern	Rule
Currency	PKR, integers only. Format as <code>PKR 4,990</code> using <code>Intl.NumberFormat("en-PK")</code> . No decimals, no paisa.
Free delivery	Orders above PKR 5,000. Show progress in the cart drawer: "PKR 1,200 away from free delivery".
Flat shipping	Configurable constant in <code>src/config/site.ts</code> — do not invent a number, leave it as a clearly-marked TODO for Umair to set.
Payment	Cash on Delivery is the default and the only live method at launch.
Phone validation	Accept <code>03XXXXXXXX</code> and <code>+923XXXXXXXX</code> . Normalise to <code>+92</code> before storing.
Provinces	Punjab, Sindh, Khyber Pakhtunkhwa, Balochistan, Gilgit-Baltistan, Azad Jammu & Kashmir, Islamabad Capital Territory.
Postal code	Optional — many Pakistani customers do not know theirs. Never block checkout on it.
Sizing	Ready-to-wear: XS-XXL. Unstitched: a single "Unstitched" variant. Bridal: made-to-order — replace Add to Cart with "Enquire on WhatsApp" where appropriate.
Product taxonomy	Support <i>piece count</i> (1/2/3-piece) and <i>fabric</i> (lawn, cotton, khaddar, chiffon, organza, silk, velvet) as first-class filters — these are how Pakistani customers actually shop.
Order numbers	Human-readable and phone-friendly: <code>KC-2026-00042</code> .

17. Phase Plan & Paste-Ready Claude Code Prompts

Work one phase per session. Start every session with: "Read docs/BUILD-SPEC.pdf and CLAUDE.md. Then do Phase N below." Do not let a session sprawl across phases — commit and start fresh.

Phase 0 — Foundation & port

PASTE INTO CLAUDE CODE

```
Read docs/BUILD-SPEC.pdf and CLAUDE.md fully before writing code.

PHASE 0 — Foundation.

1. Scaffold a Next.js 15 App Router app (TypeScript strict, Tailwind, src dir,
  @/* alias) at the repo root. Initialise shadcn/ui and add the primitives
  listed in Section 4.2 of the spec.
2. Implement the design tokens from Section 6.1 as CSS variables in
  globals.css and wire them into tailwind.config.ts as semantic colour names
  (kc-ink, kc-paper, kc-sand, kc-gold, etc.).
3. Set up the two font families from Section 6.2 via next/font with
  display swap, exposed as --font-display and --font-sans.
4. Create the full folder structure from Section 9 with empty index files
  where needed.
5. Create src/types/index.ts with every interface from Section 8.1 verbatim.
6. Create src/config/site.ts holding: brand strings, announcement bar
  messages, free-delivery threshold, flat shipping rate (TODO), social URLs
  (Facebook = https://www.facebook.com/KCoffical, others "#" placeholders),
  WhatsApp number (PLACEHOLDER), and support email (PLACEHOLDER).
7. Create src/config/nav.ts describing the full navigation tree from
  Section 7 as typed data — the header must render from this, not from JSX.
8. Create src/lib/format.ts with formatPKR() and discountPercent().
9. Create a .env.example. Add .env.local to .gitignore.

Do NOT build any pages or visual components yet.
When done: run npx tsc --noEmit and npm run build, then show me the file tree
and the tailwind config. Explain any deviation from the spec.
```

Phase 1 — Data layer

PASTE INTO CLAUDE CODE

```
PHASE 1 — Data layer with realistic mock data.

1. Implement the ProductRepository interface from Section 8.2 exactly.
2. Build MockProductRepository backed by src/data/products.ts,
  categories.ts, and collections.ts. It must fully support filtering,
  faceting, sorting, pagination, and text search per ProductQuery.
3. Generate 60 realistic Pakistani fashion products:
  - Women: unstitched lawn, embroidered 3-piece, ready-to-wear kurtas,
    formal wear, bridal.
  - Men: kurta shalwar, waistcoats, unstitched fabric.
  - Realistic PKR prices as integers (unstitched 2,500-12,000;
    ready-to-wear 3,500-18,000; bridal 45,000-250,000).
  - Real fabric names, piece counts, 2-4 images each, size and colour
    variants with stock levels, tags, ratings.
  - About 30% on sale, 20% new arrivals, 15% best sellers.
  - Names must be original. Do NOT copy product names from LAAM or any
    other real store.
4. Generate local placeholder images: neutral beige/sand blocks with a
  subtle "KC" monogram, at the correct aspect ratios (3:4 product,
  4:5 category, 16:9 editorial). Write a small script under scripts/ that
  produces them into public/placeholders/. Do not hotlink any external image.
5. Export a single src/lib/repositories/index.ts that selects the
  implementation. Components must import only from here.

When done: run the build, then print a table of 5 sample products and
demonstrate 3 example queries (filter by size, sort by price, search).
```

Phase 2 — Layout shell

PHASE 2 – Header, navigation, footer.

Build per Sections 10.2 and 11.1:

- AnnouncementBar (rotating, dismissible, from config)
- Header: sticky, transparent-over-hero on homepage only, solid after 80px scroll; logo centre-left, nav centre, actions right (search, account, wishlist with count, bag with count)
- MegaMenu: hover-intent + keyboard accessible, three link columns plus one promo image tile, driven entirely by src/config/nav.ts
- MobileNav: full-height sheet, accordion tree, bottom-pinned account links
- Header hides on scroll down / reveals on scroll up (mobile only)
- Footer: four columns, social row, payment and delivery icons, copyright

Rules: no hardcoded nav links in JSX. Focus trap and Escape on all overlays. Only one overlay open at a time (use the ui-store). Test at 360, 768, 1024, 1440 and report what you checked.

Phase 3 — Product card & homepage

PHASE 3 – ProductCard, then the full homepage.

1. Build ProductCard exactly to the contract in Section 10.1, plus ProductCardSkeleton, PriceBlock, ProductBadge, WishlistButton, QuickAddButton, and ProductGrid / ProductCarousel (embla).
2. Build the homepage with all 12 sections in the order given in Section 11.1, pulling data from the repository in a Server Component.
3. The hero image is the LCP element: priority, correct sizes, no layout shift. No scroll-triggered animations anywhere.
4. Write original KC copy for the hero, editorial banners, and newsletter. Calm and confident, short sentences, no exclamation marks. Do not use any LAAM wording.

When done: report Lighthouse mobile scores for the homepage and the transferred JS size. If LCP is above 2.5s, fix it before telling me it's done.

Phase 4 — PLP and filtering

PHASE 4 – Category pages, filters, sorting.

Build per Section 11.2:

- /women, /men, /unstitched, /ready-to-wear, /bridal, /new-arrivals, /sale, /women/[subcategory], /collections/[slug]
- FilterSidebar (desktop) and FilterDrawer (mobile) sharing one state source
- All filter and sort state in URL search params – shareable, back-button safe
- ActiveFilters chips with individual remove and clear-all
- SortDropdown with the five sort options
- Numbered pagination with rel next/prev
- Breadcrumbs with BreadcrumbList JSON-LD
- Empty state with a one-tap clear
- Grid: 2 / 3 / 4 columns at mobile / tablet / desktop
- /sale gets the distinct dark treatment from Section 11.1 item 9

Facet counts must be real, computed from the repository.

Show me the URL for "women, size M and L, ivory, sorted price ascending, page 2" and confirm a hard refresh restores that exact view.

Phase 5 — PDP

PHASE 5 – Product detail page.

Build `/products/[slug]` per Section 11.3, as a Server Component with `generateStaticParams` and `generateMetadata`, ISR 15 minutes.

- ProductGallery: desktop thumbnail rail + click-to-zoom; mobile swipeable full-bleed carousel with dots
- ProductInfo column, sticky on desktop, in the exact order specified
- ColorSelector, SizeSelector (pills; unavailable sizes struck through and disabled, never hidden), quantity stepper
- Size Guide dialog with a real Pakistani sizing table (XS-XXL, chest/waist in inches) – mark measurements as indicative
- Add to Cart + Buy Now + wishlist, trust row (COD, easy exchange, delivery estimate)
- Accordion: Description, Details & Fabric, Size Guide, Shipping, Exchange
- You May Also Like (8) and Recently Viewed (localStorage, max 12)
- StickyAddToCart on mobile once the main button scrolls away
- Product + Offer + AggregateRating + BreadcrumbList JSON-LD
- Bridal products with made-to-order flag: swap Add to Cart for an "Enquire on WhatsApp" CTA using the placeholder number

Then view-source the page and confirm the product name, price, and JSON-LD are present in the raw HTML.

Phase 6 — Cart, wishlist, search

PHASE 6 – Cart, wishlist, search.

1. Zustand cart store per Section 12, persisted, hydration-safe (no SSR mismatch – gate the count badges behind a mounted check).
2. CartDrawer + `/cart` page per Section 11.4, including the free-delivery progress bar, undo-remove toast, and empty state.
3. Wishlist store + `/wishlist` page. Guest list in localStorage. Move-to-cart opens a size picker when needed.
4. SearchModal (opens on click and on the "/" key, debounced 200ms) with recent searches, popular searches, matching categories, and up to 6 matching products with thumbnails.
5. `/search?q=` full results page reusing the PLP grid and filters.
6. Add-to-cart and wishlist animations per Section 6.5, plus an aria-live region for screen readers.

Dynamic-import CartDrawer, SearchModal, FilterDrawer, QuickViewDialog. Report the homepage JS bundle size before and after this phase.

Phase 7 — Checkout & order tracking

PHASE 7 – Checkout, orders, tracking.

Build per Section 11.5 and 11.6:

- `/checkout` single page, three sections, sticky order summary
- react-hook-form + zod validation; Pakistani phone validation and normalisation to +92; province select; optional postal code
- PaymentProvider interface from Section 11.5. Implement CodProvider fully. Card and Bank Transfer render as disabled "Coming soon".
- Order creation in a server route (`app/api/orders/route.ts`). Recompute every price server-side from the repository – never trust client totals.
- Order number format KC-YYYY-NNNNN
- Order confirmation page with the summary and the order number
- `/track-order`: order number + phone/email, server-side lookup only, six-step status timeline

Absolutely no payment credentials, keys, or provider SDKs in this phase. noindex on `/checkout`, `/cart`, `/wishlist`, `/account`.

Phase 8 — Supabase migration

PHASE 8 – Move from mock data to Supabase.

1. Apply the schema in Section 8.3 as a migration file under supabase/.
2. Enable RLS on every table with the policies described in Section 8.3. Public read only on active catalogue rows. Orders never readable anonymously.
3. Implement SupabaseProductRepository against the SAME interface. Do not change a single component.
4. Write a seed script that pushes the 60 mock products into Supabase, uploading placeholder images to Storage.
5. Supabase Auth for login, register, forgot password; account pages; merge the guest wishlist into the account wishlist on login (merge, never overwrite).
6. Flip the repository selector via env var. Verify every page still works with the mock repo too – both implementations must stay valid.

Server-side Supabase client for Server Components, browser client only where genuinely needed. Service role key is server-only and never NEXT_PUBLIC_.

Phase 9 — SEO, performance, launch

PHASE 9 – SEO, performance, accessibility, launch readiness.

1. Implement everything in Section 13: metadata on every route, canonical URLs, Open Graph images, all JSON-LD types, app/sitemap.ts generated from the repository, app/robots.ts.
2. Meet every number in the Section 14 performance budget. Run the bundle analyzer and report before/after.
3. Work through the Section 15 accessibility checklist and fix everything. Run an axe audit and report the results.
4. Build the content pages: about, contact, faqs, shipping, returns, privacy, terms, refund-policy. Write original KC copy. For anything requiring real business details (address, phone, timelines, refund windows), insert a clearly marked PLACEHOLDER – do not invent them.
5. Add a 404 page, a loading skeleton for every route, and an error boundary.
6. Produce docs/LAUNCH-CHECKLIST.md covering: env vars on Vercel, domain, Search Console, sitemap submission, Meta Pixel slot, GA4 slot, and the list of placeholders still needing real values.

Then run a full Lighthouse pass on homepage, a category page, and a product page. Report all four scores for each. Do not claim done until mobile Performance >= 90 and SEO = 100.

18. Definition of Done

Apply to every phase before it is considered complete.

- `npx tsc --noEmit` clean. `npm run build` passes. `npm run lint` clean.
- Verified at 360px, 768px, 1024px, 1440px — and mobile is a purpose-built layout.
- Keyboard traversal works; focus is visible; Escape closes overlays.
- No console errors or warnings, no hydration mismatches.
- No layout shift when images load.
- No product data hardcoded in a component.
- No invented business facts — placeholders are clearly labelled.
- Committed with a descriptive message and pushed.

19. Assets & Content Umair Must Supply

The build can reach launch-ready with placeholders, but these are the blockers for going live. Collect them in parallel with Phases 1–6.

Item	Notes
KC logo	SVG preferred, plus a monochrome version for the footer and a square mark for favicon/OG
Hero imagery	2–3 editorial shots, minimum 2400px wide, one portrait crop for mobile
Product photography	Minimum 3 images per product on a consistent background, 3:4 crop. This is the single biggest quality lever on the whole site.

Category images	Six 4:5 images for the Shop by Category cards
Real catalogue	Product names, descriptions, fabrics, piece counts, sizes, colours, stock, prices
Social URLs	Instagram and TikTok handles (Facebook is known)
Business details	WhatsApp number, support email, shop address, business hours
Policies	Actual shipping rate, delivery timelines, exchange window, refund terms
Domain	Confirm the production domain before Phase 9 (canonicals and sitemap depend on it)

On product photography: you already built an n8n + KIE.ai pipeline that turns raw product photos into consistent packshots and model shots with Supabase Storage output. Point that same pipeline at the KC catalogue. It will do more for how premium this site feels than any amount of front-end polish.

20. Guardrails — What Claude Code Must Never Do Here

1. Never fetch, scrape, copy, or mimic LAAM's assets, copy, markup, or class names.
2. Never invent a phone number, address, price, delivery timeline, refund window, or social URL. Use a labelled PLACEHOLDER and tell the user.
3. Never hardcode product data inside a component.
4. Never commit secrets, and never prefix a server-only key with `NEXT_PUBLIC_`.
5. Never trust client-supplied prices or totals — recompute server-side at checkout.
6. Never add a dependency without asking first.
7. Never ship a page without metadata.
8. Never scale down the desktop layout and call it mobile.
9. Never mark a phase done without running the build.
0. Never expand scope mid-phase. Note the idea, finish the phase, then discuss.

Suggested build order recap: Foundation → Data → Layout → Homepage → PLP → PDP → Cart/Search → Checkout → Supabase → SEO & launch. Premium visual quality and the homepage come first; everything else is scaffolding around them.